Executors and Thread Pools

9 min read · Updated 28 Jul 2026 · Topic 41 of 65

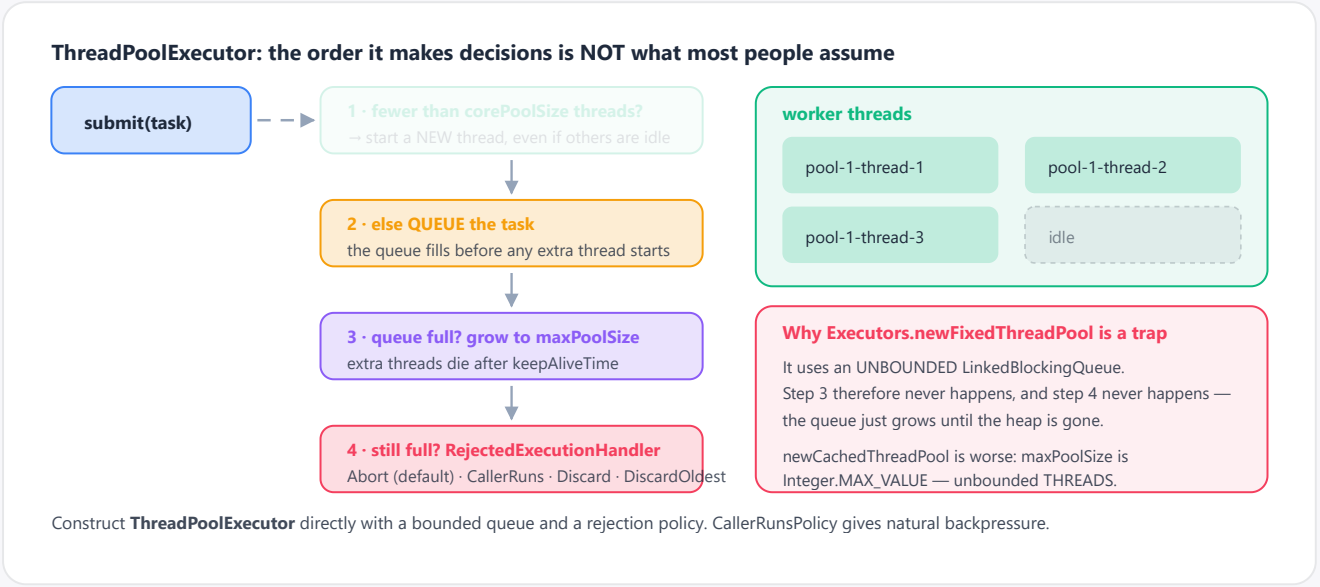
Why it matters

Nobody creates threads by hand in production code — they submit tasks to a pool. And the pool's configuration is where real outages come from: the two most convenient factory methods in the JDK both create pools that can take a server down under load, in ways the method names give no hint of. This is one of the most directly practical topics in the kit.

The concept, simply

An `ExecutorService` separates *what work to do* from *which thread does it*. You submit tasks; the pool reuses a bounded set of threads, so you pay the ~1 MB-per-thread creation cost once rather than per task.

The part almost everyone gets wrong is the order in which `ThreadPoolExecutor` makes decisions:



How it works

The six constructor parameters, and what each actually controls:

Parameter	Meaning
`corePoolSize`	Threads kept alive even when idle
`maximumPoolSize`	Upper bound — **only reached when the queue is full**
`keepAliveTime`	How long above-core threads survive idle
`workQueue`	Where tasks wait. **Bound it.**
`threadFactory`	Names threads — invaluable in dumps
`handler`	What happens when full: Abort, CallerRuns, Discard, DiscardOldest

Sizing: for CPU-bound work, roughly the number of cores. For I/O-bound work, threads spend most of their time waiting, so the pool can be much larger — the standard formula is `cores × (1 + waitTime/serviceTime)`. Measure rather than guess; a pool sized for CPU work will leave a service massively under-utilised when the real bottleneck is a database.

Code examples

What not to use, and why:

JAVA

Copy

```java
// ✗ Unbounded QUEUE. Under a burst, tasks pile up until OutOfMemoryError.
//   The name suggests the danger is bounded. It is not.
ExecutorService a = Executors.newFixedThreadPool(10);

// ✗ Unbounded THREADS — maximumPoolSize is Integer.MAX_VALUE. A traffic
```

```
// ✓ Explicit and bounded in both dimensions, with a rejection policy.
ExecutorService pool = new ThreadPoolExecutor(
    10, 20,                                  // core, max
    60L, TimeUnit.SECONDS,                   // keep-alive for extra threa
    new ArrayBlockingQueue<>(500),           // BOUNDED queue
    new ThreadFactoryBuilder().setNameFormat("order-worker-%d").build(),
    new ThreadPoolExecutor.CallerRunsPolicy() // backpressure: the submitt
);                                            // thread runs the task itse
                                              // which naturally slows int
```

Shutdown, which is routinely done wrong:

JAVA Copy

```
// ✗ shutdown() only stops NEW submissions — it returns immediately and do
//   not wait. Code after it runs while tasks are still executing.
pool.shutdown();

// ✓ The correct sequence, from the ExecutorService javadoc:
pool.shutdown();                                        // no new tasks
try {
    if (!pool.awaitTermination(30, TimeUnit.SECONDS)) {
        pool.shutdownNow();                             // interrupt running
        if (!pool.awaitTermination(30, TimeUnit.SECONDS)) {
            log.error("pool did not terminate");
        }
    }
} catch (InterruptedException e) {
    pool.shutdownNow();
    Thread.currentThread().interrupt();                 // restore the flag
}
// ⚠ shutdownNow() interrupts tasks — it cannot stop one that ignores
//   interruption. A task with an uninterruptible loop will run forever.
```

The silent-failure trap, which is the most common executor bug:

JAVA Copy

```java
executor.submit(() -> { throw new RuntimeException("boom"); });

// ✓ Either inspect the Future…
Future<?> f = executor.submit(task);
try { f.get(); } catch (ExecutionException e) { log.error("task failed", e

// ✓ …or use execute(), where an uncaught exception reaches the thread's
//   UncaughtExceptionHandler instead of being swallowed.
executor.execute(() -> { throw new RuntimeException("boom"); });

// ✓ Best: never let it escape the task.
executor.submit(() -> {
    try { doWork(); }
    catch (Exception e) { log.error("work failed", e); }
});
```

Scheduling, and the pitfall in it:

JAVA Copy

```java
ScheduledExecutorService s = Executors.newScheduledThreadPool(2);
s.scheduleAtFixedRate(this::poll, 0, 10, TimeUnit.SECONDS);    // every 10s
s.scheduleWithFixedDelay(this::poll, 0, 10, TimeUnit.SECONDS); // 10s AFTE

// ⚠ If a scheduled task throws, it is silently CANCELLED and never runs
//   again — the single most common "why did my scheduler stop?" bug.
//   Always wrap the body in try/catch.
s.scheduleAtFixedRate(() -> {
    try { poll(); } catch (Exception e) { log.error("poll failed", e); }
}, 0, 10, TimeUnit.SECONDS);
```

Under the hood

Why the queue fills before the pool grows is the design decision that surprises people, and it is deliberate: growing threads is expensive, so the executor prefers to queue. The

`CallerRunsPolicy` **is the most useful rejection handler** because it provides backpressure rather than failure: when the pool and queue are full, the submitting thread executes the task itself. That thread is then not submitting more work, so intake naturally slows to match capacity. For a request-handling service that is usually better than dropping work or throwing.

Name your threads. The default names are `pool-1-thread-3`, which tells you nothing in a dump with four pools. A `ThreadFactory` that names them after their purpose turns thread-dump analysis from guesswork into reading.

Virtual threads (Java 21) change the advice for I/O-bound work. `Executors.newVirtualThreadPerTaskExecutor()` creates a new virtual thread per task — no pooling, because they are cheap — so the whole exercise of sizing a pool for blocking I/O disappears. Pools remain the right model for CPU-bound work, where the limit is cores rather than thread cost. Knowing which of those two situations you are in is the modern version of this question.

Common mistakes

- `newFixedThreadPool` **in production.** Unbounded queue → `OutOfMemoryError` under load.
- `newCachedThreadPool` **in production.** Unbounded threads → the machine dies.
- **Expecting** `maximumPoolSize` **to matter with an unbounded queue.** It never triggers.
- **Assuming** `shutdown()` **waits.** It does not — use `awaitTermination`.
- **Ignoring the** `Future`**.** Exceptions from `submit` vanish silently.
- **Unguarded scheduled tasks.** One exception cancels the schedule permanently.
- **Unnamed threads.** Makes production dumps far harder to read.
- **Blocking calls in the common ForkJoinPool.** Chapter 7's parallel-stream problem.
- **One pool for everything.** Slow tasks starve fast ones; isolate by workload.

Interview lens

A: In order: if fewer than `corePoolSize` threads exist it starts a new one even if others are idle; otherwise it queues the task; only if the queue is full does it grow toward `maximumPoolSize`; and if that also fails, the rejection handler runs. The counter-intuitive part is that queueing comes before growing, which means `maximumPoolSize` never takes effect with an unbounded queue.

Q: "What is wrong with `Executors.newFixedThreadPool`?"

A: It uses an unbounded `LinkedBlockingQueue`. Under a burst the queue grows without limit until the heap is exhausted, and because the queue never fills, the rejection policy never runs — so there is no backpressure signal at all. `newCachedThreadPool` has the mirror problem: `maximumPoolSize` is `Integer.MAX_VALUE`, so it creates unbounded threads. I construct `ThreadPoolExecutor` directly with a bounded queue and an explicit rejection policy.

Q: "How do you size a thread pool?"

A: By what the tasks do. CPU-bound work wants roughly the number of cores, since more threads just add context switching. I/O-bound work can use many more, because threads are mostly waiting — the usual formula is cores × (1 + wait time / service time). I measure rather than guess. And on Java 21, for I/O-bound work I would use virtual threads and stop sizing a pool at all.

Q: "How do you shut an executor down properly?"

A: `shutdown()` to stop new submissions, then `awaitTermination` with a timeout, then `shutdownNow()` if it has not finished, then await again. `shutdown()` alone returns immediately and does not wait, which is the usual mistake. And `shutdownNow()` only interrupts — a task that ignores interruption will keep running.

A: With `submit()`, the exception is captured in the `Future` and surfaces only when someone calls `get()`, wrapped in `ExecutionException` — so if nobody checks, it disappears entirely. With `execute()`, it propagates to the thread's uncaught-exception handler. The safest pattern is to catch inside the task so failures are always logged.

Q: "Which rejection policy would you choose?"
A: `CallerRunsPolicy` for most server workloads, because it gives backpressure rather than failure — the submitting thread runs the task, so it stops submitting and intake slows to match capacity. `AbortPolicy`, the default, is right when the caller can handle rejection meaningfully. The discard policies are rarely acceptable, since they lose work silently.

Cheat-card

- Order: **core threads → queue → grow to max → reject**. Queue fills *before* the pool grows.
- **Unbounded queue ⇒ `maximumPoolSize` is dead code.**
- Avoid `newFixedThreadPool` (unbounded queue) and `newCachedThreadPool` (unbounded threads).
- Construct `ThreadPoolExecutor` with a **bounded queue** + `CallerRunsPolicy` for backpressure.
- Size: CPU-bound ≈ cores; I/O-bound ≈ cores × (1 + wait/service). **Measure.**
- Shutdown: `shutdown()` → `awaitTermination` → `shutdownNow()` → await. `shutdown()` does **not** wait.
- `submit()` hides exceptions in the `Future`; `execute()` surfaces them.
- A scheduled task that throws is **cancelled forever** — always try/catch inside.
- **Name your threads.** Isolate pools by workload.
- Java 21 I/O-bound → `newVirtualThreadPerTaskExecutor()`, no sizing needed.

1. In what order does ThreadPoolExecutor handle a submitted task?

- ◯ It grows to maximumPoolSize first, then queues
- ◯ Core threads first, then QUEUE the task, then grow toward maximumPoolSize only when the queue is full, then reject
- ◯ It always queues first, then creates a thread per queued task
- ◯ It rejects immediately if all core threads are busy

Check answer

2. What is dangerous about Executors.newFixedThreadPool in production?

- ◯ It cannot be shut down
- ◯ It creates too many threads
- ◯ It uses an UNBOUNDED LinkedBlockingQueue, so a burst grows the queue until the heap is exhausted — and because the queue never fills, there is no backpressure signal at all
- ◯ It never reuses threads

Check answer

3. A task submitted with submit() throws an exception and nobody calls get(). What happens?

- ◯ It propagates to the thread's uncaught exception handler
- ◯ The executor shuts down

It is printed to stderr automatically

- ○ The exception is captured in the Future and never surfaces — it disappears entirely

Check answer

‹ Previous
Deadlock, Livelock and Starvation

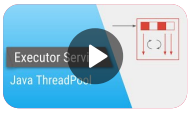
Next ›
CompletableFuture and Asynchronous Composition

LEARN MORE

ARTICLES & DOCS

- ThreadPoolExecutor — API docs
  docs.oracle.com
- Executors — API docs
  docs.oracle.com

VIDEOS



- Java ExecutorService - Part 1 - Introduction
  Defog Tech



- Java Threads Tutorial – Thread Pools & Executors Explained with Examples | Fixed, Cached, Scheduled
  Dev Portal

Hand-picked references for this lesson.